CHAPTER 5A: Server-Side Language
PHP Basics (A)

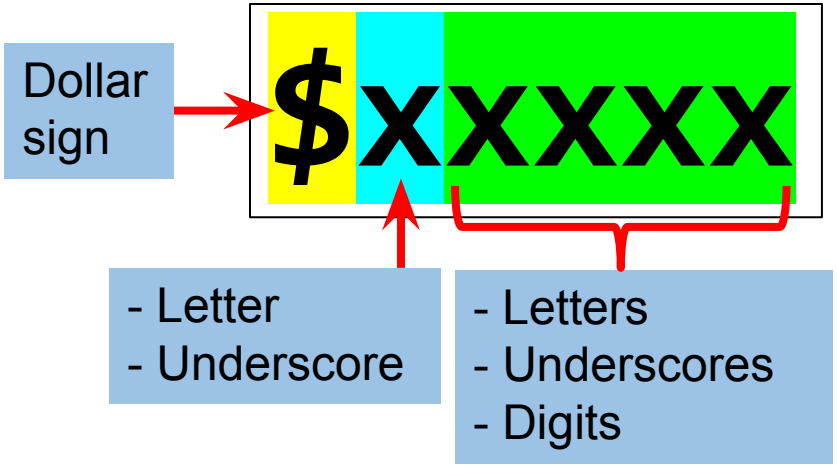
Topics covered:-

- **Variables**
- **Data Types**
- **PHP Operators**
- **Constants**

Variables

NOTE: PHP function name is case insensitive, however

- PHP variable names are **case sensitive**.
- PHP variable naming rules:-
  - Begin with a **dollar sign ($)**.
  - Followed by a **letter** or an **underscore**.
  - Then optionally followed by any number of **letters**, **underscores** or **digits**.
- <u>Examples:-</u>
  - **`$a`**
  - **`$_`**
  - **`$student_id`**
  - **`$_db`**
  - **`$_123`**



Declaring and Initializing a variable

Start of PHP code block

```
<?php
/* Example:
   Declare and initialize variable */

$a = 1;     // Integer
$b = 1.2;   // Float
$c = 'A';   // String
$d = true;  // Boolean
?>
```

PHP block comment

PHP line comment

End of PHP code block

Optional if this is the last line of file

Data Types

- There are **3 general types of data:-**
  [1.] Scalar Data Types
  [2.] Compound Data Types
  [3.] Special Data Types

- **Scalar Data Types** can be divided into 4 sub types:-
  [1.] Integer
  [2.] Float
  [3.] String
  [4.] Boolean
- **Compound Data Types** can be divided into 2 sub types:-
  [1.] Array
  [2.] Object
- **Special Data Types** can be divided into 2 sub types:-
  [1.] Resource
  [2.] Null

Data Types

- **Scalar** means data that contains a **single value**.

- **Compound** means data that contains **more than one value**.

- **Special** means neither scalar nor compound but they have **specific meaning**.

  - **Resource** □ file pointer, database connection, etc.
  - **Null**

PHP Basics (A)

Scalar Data Types	Description	Example
Integer	A whole number	15
Float	A floating-point number	3.5
String	A series of characters	“Hello, World”
Boolean	Represents either true or false	true

- PHP does not differentiate between **String** and **Char**.
- PHP does not differentiate between **Float** and **Double**.

PHP Basics (A)

Compound Data Types	Description
Array	An list of values referenced by names or index
Object	A type that contains properties and methods

Special Data Types	Description
Resource	Contains reference to an external resource, such as file or database
Null	Contain null as a value

Loose Data Type

- No need to specify data type when declaring PHP variable.
- In C, C++ or Java:-

  int value = 3; → Strongly-typed

- In PHP:-

  $value = 3; → Loosely-typed

- PHP converts a variable's data type automatically depending on the value assigned.

```
$a = 123; // $a is integer
$a = 'A'; // $a is string now
```

Loose Data Type

- **Advantages**:-
  - Very flexible in declaring variables.
  - The same variable can easily be used in different situations.
  - Programmers do not have to worry about the type of variable when declaring them.

- **Disadvantages**:-
  - It is rather difficult to identify if an error occurs due to passing a floating-point value to a piece of code that expects to be working on an integer value.

Data Type Testing Function

Function	Description
is_int(value) is_integer(value)	Returns true if value is an integer.
is_float(value) is_double(value)	Returns true if value is a float or double.
is_string(value)	Returns true if value is a string.
is_bool(value)	Returns true if value is a bool.
is_array(value)	Returns true if value is an array.
is_object(value)	Returns true if value is an object.
is_resource(value)	Returns true if value is a resource.
is_null(value)	Returns true if value is null.

gettype() Function

- Return the data type of a value as string.

```
$a = 1;
$b = 1.2;
$c = 'A';
$d = true;

gettype($a); // 'integer'
gettype($b); // 'double'
gettype($c); // 'string'
gettype($d); // 'boolean'
```

settype() Function

- Change the data type of a variable.

```
$a = 5.6;                    // $a = 5.6 (double)
settype($a, 'integer'); // $a = 5   (integer)
```

Support type strings:
- **'boolean'** or **'bool'**
- **'integer'** or **'int'**
- **'double'** or **'float'**
- **'string'**
- **'array'**
- **'object'**
- **'null'**

Note that fractional digits are truncated

Convert Data Type - Casting

- Other than the function **settype()**, PHP **type casting** is also available. Example:-

```
$a = '5.6ABC';

(boolean)$a; // true
   (bool)$a; // true
(integer)$a; // 5
    (int)$a; // 5
 (double)$a; // 5.6
  (float)$a; // 5.6
 (string)$a; // '5.6ABC'
  (array)$a; // ['5.6ABC']
 (object)$a; // {scalar:'5.6ABC'}
```

The following values are casted to **false**

```
false
0
0.0
''
'0'
[]
null
```

Other values are **true**

PHP does not support this object notation □ put here for explanation only

Type casting does not alter the original variable

Test your casting skill…

- Assuming the initial value of the variable **$x** is **8.15**

Statement using type casting	Value (after casting)	Output (after echo)	Alternative function
`echo (int)$x;`	`8`	**8**	`intVal($x)`
`echo (string)$x;`	`'8.15'`	**8.15**	`strVal($x)`
`echo (float)$x;`	`8.15`	**8.15**	`floatVal($x)`
`echo (boolean)$x;`	`true`	**1**	`boolVal($x)`

echo converts value to **string** before output.
- **true** is converted to **'1'**
- **false** and **null** is converted to **''** (empty string)

Operators and Expressions

- Operators is a symbol used to manipulate one or more values.
- Operators in PHP can be grouped into **10 types**:-
  - Arithmetic
  - Assignment
  - Incrementing and Decrementing
  - Comparison
  - Logical
  - String
  - Bitwise
  - Error Control
  - Execution
  - Array

Skip

Operator TYPE 1 - *Arithmetic*

Arithmetic Operators	Example Equation
+ (addition)	8 + 2 = 10
- (subtraction)	8 – 2 = 6
* (multiplication)	8 * 2 = 16
/ (division)	8 / 2 = 4
% (modulus)	8 % 2 = 0 (remainder)
** (exponentiation)	8 ** 2 = 8^2 = 64

```
$a = 3 / 2;          // 1.5
$b = intdiv(3, 2); // 1
```

Use **intdiv()** function for integer division

Operator TYPE 2 - *Assignment*

Assignment Operators	Expression	Evaluated Expression and its values
=	$a = $b = 8.15	$a ⍰ 8.15, $b ⍰ 8.15
+=	$a += 2	$a = $a + 2
-=	$a -= 2	$a = $a – 2
*=	$a *= 2	$a = $a * 2
/=	$a /= 2	$a = $a / 2
%=	$a %= 2	$a = $a % 2
**=	$a **= 2	$a = $a ** 2
.=	$a = 'ABC' $a .= 'DEF'	$a = $a . 'DEF' $a ⍰ 'ABCDEF'
??=	$a ??= 2	$a = $a ?? 2

<u>Null coalesce:</u>
Assign **2** to **$a** if **$a** is **null**

Operator TYPE 3 – *Increment and Decrement*

Increment and Decrement Operators	Example	Result
++$x (pre-increment)	$x = 5; echo ++$x;	Echo output ⍰ 6 Now $x contains 6
$x++ (post-increment)	$x = 5; echo $x++;	Echo output ⍰ 5 Now $x contains 6
--$x (pre-decrement)	$x = 5; echo --$x;	Echo output ⍰ 4 Now $x contains 4
$x-- (post-decrement)	$x = 5; echo $x--;	Echo output ⍰ 5 Now $x contains 4

PRE ⍰ Increase/decrease first, then read value
POST ⍰ Read value first, then increase/decrease

Operator TYPE 4 - *Comparison*

Comparison Operators	Example	Result
== (equal)	$x == $y	True if $x is equal to $y
!= or <> (not equal)	$x != $y $x <> $y	True if $x is not equal to $y
=== (identical)	$x === $y	True if $x is equal to $y <u>AND</u> **same data type**
!== (not Identical)	$x !== $y	True if $x is not equal to $y <u>OR</u> **different data type**

var_dump() is a debugging function that print the data type, structure and value of a variable

```
var_dump(1 ==   '1'); // bool(true)
var_dump(1 ===  '1'); // bool(false)
var_dump(1 ==  true); // bool(true)
var_dump(1 === true); // bool(false)
```

Operator TYPE 4 - *Comparison*

Comparison Operators	Example	Result
< (less than)	$x < $y	True if $x is less than $y
> (greater than)	$x > $y	True if $x greater than $y
<= (less than or equal to)	$x <= $y	True if $x is less than or equal to $y
>= (greater than or equal to)	$x >= $y	True if $x is greater than or equal to $y
<=> (spaceship)	$x <=> $y	*** Used in comparison function *** $x < $y ⍰ Return negative integer (e.g. -1) $x == $y ⍰ Return zero (0) $x > $y ⍰ Return positive integer (e.g. +1)

Operator TYPE 4 - *Comparison*

- **Conditional Operator (?:)**

```
$a = $b ? 1 : 2;
```

Return this if **$b** is **true**

Return this if **$b** is **false**

Equivalent…

```
if ($b) {
  $a = 1;
}
else {
  $a = 2;
}
```

- **Null Coalescing Operator (??)**

```
$a = $b ?? $c;
```

Return this if **$b** is **not null**

Return this if **$b** is **null**

Equivalent…

```
if ($b !== null) {
  $a = $b;
}
else {
  $a = $c;
}
```

Operator TYPE 5 - *Logical*

Logical Operators	Example	Result
&& (and)	$x && $y	True if both $x and $y evaluated to true
|| (or)	$x || $y	True if either $x or $y evaluated to true
xor (exclusive or)	$x xor $y	True if $x or $y (but not both) evaluated to true
! (not)	!$x	True if $x is false

- PHP also support **and** and **or** operators ⍰ seldom used
- Similar to **&&** and **||** but with very low precedence.
- **$x and $y**
- **$x or $y**

Operator TYPE 6 - *String*

String Concatenation / Dot Operator	Example	Result
. (dot)	$x . $y	$x = 123 $y = 'ABC' Result ⇒ '123ABC'

NOTE: Values are auto converted to string before string concatenation

null ⇒ **''** (empty string)
false ⇒ **''** (empty string)
true ⇒ **'1'**

-01 = -1 ⇒ **'-1'** (string)
002 = 2 ⇒ **'2'** (string)
0.0 = 0 ⇒ **'0'** (string)

```
  1 . 234   // '1234'
-01 . 002   // '-12'
0.0 . 1.2   // '01.2'
'A' . true  // 'A1'
'A' . false // 'A'
'A' . null  // 'A'
```

Operators Precedence

- An operator with a higher precedence get executed first.

Precedence of some PHP operators (Highest First)
** (exponentiation)
++ -- (increment / decrement)
(int) (float) (string) (array) (object) (bool) (casting)
! (not)
* / % (arithmetic)
+ - . (arithmetic)
< <= > >= (comparison)
== != === !== <> <=> (comparison)
&& (and)
|| (or)
?? (null coalescing)
?: (conditional operator)
= += -= *= **= /= .= %= ??= (assignment)
and
xor
or

Constants

- Once constant is defined, its **value cannot be changed**.
- Same naming rules as variable EXCEPT **constant DOES NOT start with a dollar sign ($).**
- It is advisable to use **all uppercase** for constant name to differentiate it from variable name.

Constant name (string)

```
define('PI', 3.14159);
define('NUMBERS', [1, 2, 3]);
define('DB', new PDO(...));
define('SUM', $a + $b);

$area = PI * $radius ** 2;
```

Scalar value

Array

Object

Run-time expression

Using constant value

Constants

- Can also define constant using the **const** keyword.

```
const PI = 3.14159;
$area = PI * $radius ** 2;
```

- <u>Differences:</u>
  - **const** keyword can only be used in **global scope** or **class** (e.g. cannot be defined in function or if-else block).
  - **define(…)** is run-time. **const** is compile-time. The value assigned to **const** must be determinable at compile-time.

define(…) OK

```
$a = 12;
$b = 34;
const X = 12 + 34; // OK
const Y = $a + $b; // FAILED
```

define(…) can handle this

CHAPTER 5A: Server-Side Language
PHP Basics (A)

Topics covered:-

- **IF and IF-ELSE statement**
- **SWITCH statement and MATCH expression**
- **Loops**
- **Date and time functions**

IF and IF-ELSE statements

Similar to C and C++

```
if ($a == 1) {
  $b = 'A';
}
```

```
if ($a == 1) {
  $b = 'A';
}
else {
  $b = 'X';
}
```

```
$b = $a == 1 ? 'A' : 'X';
```

Rewritten using **conditional operator**

```
if ($a == 1) {
  $b = 'A';
}
else if ($a == 2 || $a == 3) {
  $b = 'B';
}
else {
  $b = 'X';
}
```

$a	$b
1	'A'
2, 3	'B'
Others	'X'

SWITCH statement and MATCH expression

Similar to C and C++

New in PHP 8.0.0

```
switch ($a) {
  case 1:
    $b = 'A';
    break;
  case 2:
  case 3:
    $b = 'B';
    break;
  default:
    $b = 'X';
    break;
}
```

$a	$b
1	'A'
2, 3	'B'
Others	'X'

```
$b = match ($a) {
  1       => 'A',
  2, 3    => 'B',
  default => 'X',
};
```

Only allow **ONE (1)** statement each

MATCH Expression:

- More concise compare to **switch** statement.
- **switch** matches values using value-check (==).
- match matches values using identity-check (===)
  - i.e. data type must be the same also.

NOTE:

With some clever arrangement, we can use **switch** statement and **match** expression for range comparison too:

if-else statement

```
if      ($mark >= 80) { $grade = 'A'; }
else if ($mark >= 65) { $grade = 'B'; }
else if ($mark >= 50) { $grade = 'C'; }
else                  { $grade = 'F'; }
```

switch statement

```
switch (true) {
  case $mark >= 80: $grade = 'A'; break;
  case $mark >= 65: $grade = 'B'; break;
  case $mark >= 50: $grade = 'C'; break;
  default         : $grade = 'F'; break;
}
```

match expression

```
$grade = match (true) {
  $mark >= 80 => 'A',
  $mark >= 65 => 'B',
  $mark >= 50 => 'C',
  default     => 'F',
};
```

Loops

Similar to C and C++

while loop

```
$i = 1;
while ($i <= 5) {
  echo $i;
  $i++;
}
// 1 2 3 4 5
```

do-while loop

```
$i = 1;
do {
  echo $i;
  $i++;
} while ($i <= 5);
// 1 2 3 4 5
```

for loop

```
for ($i = 1; $i <= 5; $i++) {
  echo $i;
}
// 1 2 3 4 5
```

FOREACH loops

Specifically for the use with **arrays**:

```
$arr = ['A', 'B', 'C'];
foreach ($arr as $v) {
  echo $v;
}
// A B C
```

Access values only

```
$arr = ['A', 'B', 'C'];
foreach ($arr as $k => $v) {
  echo "$k$v";
}
// 0A 1B 2C
```

Access both keys (index) and values

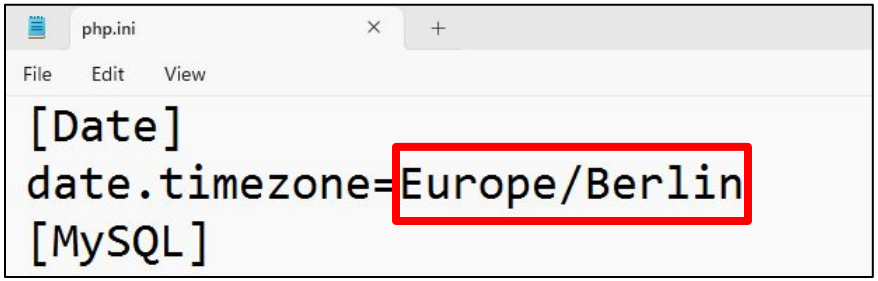
Using Date and Time functions

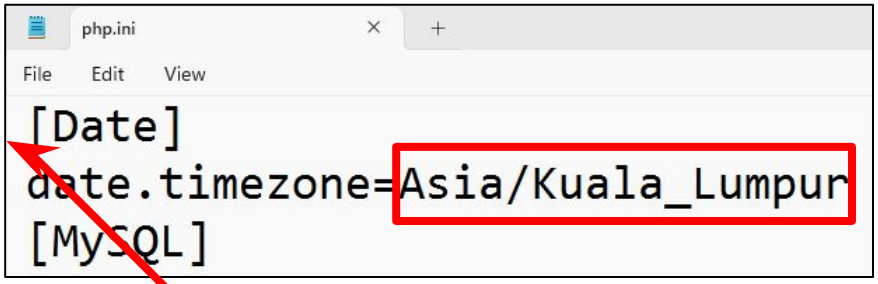
- The default time zone for XAMPP PHP is **'Europe/Berlin'**.
- Set default time zone using the following codes (before calling any date and time functions):

```
date_default_timezone_set('Asia/Kuala_Lumpur');
```

The time zone id

- List of supported time zones: https://www.php.net/manual/en/timezones.php
- Can also modify **php.ini** file directly for global changes:





Date and Time Functions

Function	Description
checkdate()	A date is valid if: ✔ month is between 1 and 12 inclusive ✔ day is within the allowed number of days for the particular month ✔ year is between 1 and 32767 inclusive
date_default_timezone_get()	Returns the default time zone
date_default_timezone_set()	Sets the default time zone
date()	Formats a local time/date
getdate()	Returns an array that contains date and time information for a Unix timestamp [seconds] - seconds ,[minutes] - minutes , [hours] - hours [mday] - day of the month , [wday] - day of the week , [year] - year [yday] - day of the year , [weekday] - name of the weekday , [month] - name of the month
localtime()	Returns an array that contains the time components of a Unix timestamp
time() / mktime()	Returns the current timestamp.

Date Function Formatting

```php
echo date('Y-m-d');
// 2024-08-25

echo date('d F Y');
// 25 August 2024

echo date('l');
// Sunday

echo date('H:i:s');
// 18:30:59

echo date('h:i a');
// 06:30 pm
```

Character	Meaning	Example
Y	year as 4 digits	2024
y	year as 2 digits	24
n	month as 1 or 2 digits	8
m	month as 2 digits	08
F	month (full name)	August
M	month as 3 letters abbr.	Aug
j	day of the month as 1 or 2 digits	25
d	day of the month as 2 digits	25
l	day of the week (full name)	Sunday
D	day of the week as 3 letters abbr.	Sun
g	hour, 12-hour format as 1 or 2 digits	6
G	hour, 24-hour format as 1 or 2 digits	18
h	hour, 12-hour format as 2 digits	06
H	hour, 24-hour format as 2 digits	18
i	minutes as 2 digits	00
s	seconds as 2 digits	00
a	am or pm lowercase	am
A	AM or PM uppercase	AM

References:

- Date and time functions:
  https://www.php.net/manual/en/ref.datetime.php

- Date and time formats:
  https://www.php.net/manual/en/datetime.format.php

CHAPTER 5A: Server-Side Language
PHP Basics (A)

Topics covered:-

- **Working with string**
- **Regular expression (regex)**

Reference (string functions) =
https://www.php.net/manual/en/ref.strings.php

Escape Sequence

NOTE: Escape sequence DOES NOT work in **single-quoted** string EXCEPT **\'** which represents a single-quote mark

- Use inside **double-quoted strings**.

Sequence	Meaning
`\n`	Newline / line feed character.
`\r`	Carriage return
`\t`	Horizontal tab character
`\v`	Vertical tab character
`\f`	Form feed character
`\\`	Escape backslash
`\$`	Escape dollar sign
`\"`	Escape double quote

Creating and Accessing Strings

- Differences using **single-quote** and **double-quote** to enclose a string:

```
echo 'A\tB'; // A\tB
echo "A\tB"; // A    B

$name = 'Suzy';
echo 'Hello, $name'; // Hello, $name
echo "Hello, $name"; // Hello, Suzy
```

Single-quoted string DOES NOT evaluate escape sequence and variable

Complex String Expression

```
$pet = 'Cat';
echo "I love $pets";    // FAILED
echo "I love {$pet}s"; // I love Cats
```

Because variable **$pets** is undefined

Use braces **{...}** to cover the variable if necessary

```
$fruits = ['Apple', 'Banana'];
echo "I love $fruits[0]"; // I love Apple
```

Indexed array

Associative array

```
$fruits = [
  'APP' => 'Apple',
  'BNN' => 'Banana',
];
$key = 'BNN';
echo "$fruits[APP] and $fruits[$key]"; // Apple and Banana
```

Find the Length of a String

- Use the **strlen()** function:

```
echo strlen('Hello World'); // 11
```

Accessing a Character

- Use the **index** (position):

```
$s = 'Hello World';
echo $s[0];  // 'H'
echo $s[6];  // 'W'
echo $s[-1]; // 'd'
echo $s[-2]; // 'l'
```

Negative index access a character from backward

Accessing a Substring

Start from index **0**, read **5** characters

- Use the **substr()** function:

```
$s = 'Hello World';
echo substr($s,  0, 5); // 'Hello'
echo substr($s,  6, 5); // 'World'
echo substr($s,  6);    // 'World'
echo substr($s, -5);    // 'World'
```

Offset, the start index (can be negative)

Optional **length**, default until the end

- **substr_count()** = Count the number of substring occurences

```
$s = '-XXX-XXX-XXX-';
echo substr_count($s, '-');   // 4
echo substr_count($s, '+');   // 0
echo substr_count($s, 'X');   // 9
echo substr_count($s, 'XXX'); // 3
```

Searching Strings

- **strstr()** = Find the first occurrence of a string

```
$s = 'ABCDEF';
echo strstr($s, 'CD');          // 'CDEF'
echo strstr($s, 'CD', true);    // 'AB'
echo strstr($s, 'XX');          // false
```

Return **false** if not found

Optional **true** to return the part before the search string

- This function is case-sensitive.
- For case-insensitive search, use the **stristr()** function.

Searching Strings

- **strpos()** = Find the **position** of the first occurrence of a string

```
$s = 'ABCDEF';
echo strpos($s, 'CD');     // 2
echo strpos($s, 'CD', 2); // 2
echo strpos($s, 'CD', 4); // false
```

Return **position (index)** if found

Return **false** if not found

Optional **offset** (start index)

Start from index 4 = **'EF'**
'CD' is not found in **'EF'**

- This function is case-sensitive.
- For case-insensitive search, use the **stripos()** function.

- **strrpos()** = Find the position of the last occurrence of a string.
- **strripos()** = Case-insensitive version.

Contains, starts with and ends with

```
$s = 'ABCDEF';
echo str_contains($s, '');   // true
echo str_contains($s, 'CD'); // true
echo str_contains($s, 'XX'); // false

echo str_starts_with($s, '');   // true
echo str_starts_with($s, 'AB'); // true
echo str_starts_with($s, 'XX'); // false

echo str_ends_with($s, '');   // true
echo str_ends_with($s, 'EF'); // true
echo str_ends_with($s, 'XX'); // false
```

- All these functions are **case-sensitive**.

" (empty string) returns true for all these functions

Replacing Text

- **str_replace()** = Replace all occurrences of the search string with the replacement string

Search string

Replace with

Optional variable holding the count

```
$s = 'AXBXCXD';
echo str_replace('X', '-', $s, $n); // A-B-C-D
echo $n; // 3
```

Total 3 replacements had been made

- This function is case-sensitive.
- For case-insensitive replace, use the **str_ireplace()** function.

Working with Uppercase/Lowercase

Function	Description
strtoupper()	Convert a string to all uppercase.
strtolower()	Convert a string to all lowercase.
ucfirst()	Convert the first character to uppercase.
lcfirst()	Convert the first character to lowercase.
ucwords()	Convert the first character of each word to uppercase.

Working with Uppercase/Lowercase

```
$s = 'HELLO world';
echo strtoupper($s); // HELLO WORLD
echo strtolower($s); // hello world
echo ucfirst($s);    // HELLO world
echo lcfirst($s);    // hELLO world
echo ucwords($s);    // HELLO World
```

Formatting String using printf() & sprintf() functions

Format and **print output to screen**

Format and **return output as string**

- Similar to how **printf()** and **sprintf()** in C or C++ work.
- Example of conversion specification are as follows:-
  - %d
  - %f
  - %s
  - %b
  - %c
  - %o and etc.

Type Specifiers

Type Specifier	Meaning
b	Treat the argument as an integer and format it as a binary number.
c	Treat the argument as an integer and format it as a character with that ASCII value.
d	Treat the argument as an integer and format it as a signed decimal number.
e	Format the argument in scientific notation (for example, 3.45e+2)
f	Format the argument as a floating-point number, taking into account the current locale settings (for example, many European locales use a comma for the decimal point, rather than a period.)
F	Format the argument as a floating-point number, ignore the locale.

Type Specifiers (Continue)

Type Specifier	Meaning
o	Treat the argument as an integer and format it as an octal number.
s	Format the argument as string.
u	Treat the argument as an integer and format it as a unsigned decimal number.
x	Treat the argument as an integer and format it as a lowercase hexadecimal number.
X	Treat the argument as an integer and format it as a uppercase hexadecimal number.
%	Display a literal percent (%) symbol. This doesn't require an argument.

Specifying Sign

- By default, **printf()** displays negative numbers with a minus (-) symbol. In order to print (+) sign, we need to add a sign specifier in the type specifier.

- **Example:-**

```
printf("%d" ,  123); //display  123
printf("%d" , -123); //display -123
printf("%-d", -123); //display -123
printf("%+d",  123); //display +123
printf("%+d", -123); //display -123
```

Padding the Output

- Using padding specifier.
- The purpose is to format output with leading zeros to a number or padding zeros for alignment.
- Padding specifier consists of the following two (2) :-
  - Zero (to pad with zeros)
  - Space (to pad with spaces)
- **Example:-**
- You would like to ensure the number printed is always 8 digits long.

```
printf("%08d", 123); //display 00000123
```

Note that padding does not truncate the number output, for example,

```
printf("%06d", 12345678); //display 12345678
```

However, this is not the case with string padding.

Padding the Output (Right-aligned)

- **Example:-**
- You would like to ensure the text printed is always aligned right.

```
echo "<pre>";
printf("%20s\n", "Welcome");
printf("%20s\n", "Android User");
printf("%20s\n", "in Chrome");
printf("%20s\n", "Best for All Browser");
echo "</pre>";
```

```
             Welcome
        Android User
           in Chrome
Best for All Browser
```

Note : You may put a (-) minus symbol in front of the number 20 to make it left-aligned instead of the default right-aligned.

Padding the Output (Left-aligned)

- **Example:-**
- You would like to ensure the text printed is always aligned left.

```
echo "<pre>";
printf("%-20s\n", "Welcome");
printf("%-20s\n", "Android User");
printf("%-20s\n", "in Chrome");
printf("%-20s\n", "Best for All Browser");
echo "</pre>";
```

```
Welcome
Android User
in Chrome
Best for All Browser
```

Using own padding character

- You may use your own padding character by specifying an apostrophe before using the new padding character.

```
printf("%'*50s" , 'ABC');
```

```
**************************************************ABC
```

```
printf("%'*-50s", 'ABC');
```

```
ABC**************************************************
```

Formatting Floating-point Numbers

- Formatting the number of decimal places using precision specifier.
- To use precision specifier, use a . (dot) followed by the number of decimal places to use.
- **Example:-**

```
echo "<pre>";
printf("%f\n", 12);
printf("%.3f\n", 12);
printf("%12.3f\n", 12);
printf("%010.3f\n", 12);
echo "</pre>";
```

```
12.000000
12.000
      12.000
000012.000
```

Default precision

String Padding

- **Example:-**

```
printf("%.10s", "Too good to be true");
```

- **Resulted Output:-**

Too good t

- As mentioned, string padding with the padding specifier used will truncate the output to the exact number of characters. This lead to character string losses. Be careful.

Storing Result of printf() Using sprintf()

- There are cases where you would not want to always print the result generated by **printf()** function but to keep it for later use.
- With this, **sprintf()** is used to keep the resulted value in a variable.

- **Example:-**

```
$welcome = sprintf('%.7s', 'Welcome to PHP World');
echo $welcome; //display 'Welcome'
```

Trimming strings

- **trim()** – removes white spaces at **start and end** of the string
- **ltrim()** – left trim – removes white spaces at **start** of the string
- **rtrim()** – right trim removes white spaces at **end** of the string

```
$s = '   ABC   ';
echo trim($s);  // 'ABC'
echo ltrim($s); // 'ABC   '
echo rtrim($s); // '   ABC'
```

- **Note**: You may specify the second argument for the characters to trim.

```
$s = '12345ABC67890';
echo trim($s, '1234567890');  // 'ABC'
```

Trim all digits

Formatting Numbers with number_format() function

- Format the number with **thousand separator (default = ',')** and the specified **number of decimal places (default = 0)**.

```
$n = 1234.5678;
echo number_format($n);                    // 1,235
echo number_format($n, 2);                 // 1,234.57
echo number_format($n, 2, '@', '_');  // 1_234@57
```

Number of decimal points (default = 0)

Decimal separator (default = '.')

Thousands separator (default = ',')

Regular Expression (regex)

- Regular Expressions are patterns that are used for matching and manipulating strings according to specified rules

Example:

```
if(preg_match('/^\d{6}-\d{2}-\d{4}$/', $str)) {
   ...
}
```

Start of regex

Start of string

End of string

End of regex

This pattern matches Malaysia IC number □ **123456-12-1234**

- Regular expression patterns normally consist of a combination of:-
  - ✔ **literal characters**
  - ✔ **Meta-characters**
  - ✔ **Quantifiers**
  - ✔ **Character classes**

Meta-characters

Meta-character	Description
\	Escape character
^	beginning of a string
$	the end of a string
.	Any single character except newline
|	Alternatives (or)
[	Start of a character class
]	End of a character class
(	Start of a sub-pattern / group
)	End of a sub-pattern / group
{	Start of a quantifier
}	End of a quantifier

Some characters have special meaning in regex, hence the raw characters need to be escaped. For examples:

- **\-** = dash (-)
- **\.** = dot (.)
- **** = backslash (\)
- **\/** = forward slash (/)
- **\^** = caret (^)
- **\$** = dollar ($)

Quantifiers

Character	Meaning
?	0 or 1 □ {0,1}
*	0 or more □ {0,}
+	1 or more □ {1,}
{x}	Exactly x occurrences
{x,y}	Between x and y (inclusive)
{x,}	At least x occurrences
{,y}	At most y occurrences

Character Classes

Class	Shortcut	Meaning
`[0-9]`	`\d`	Any digit
`[\f\r\t\n\v]`	`\s`	Any white space
`[A-Za-z0-9_]`	`\w`	Any word character
`[^0-9]`	`\D`	Not a digit
`[^\f\r\t\n\v]`	`\S`	Not white space
`[^A-Za-z0-9_]`	`\W`	Not a word character

- **^** symbol means NOT within character class **[…]**
- **[0123456789]** = **[0-9]** (in ASCII sequence) = **\d**
- **[A-Za-z]** = matches any uppercase and lowercase alphabets

PCRE Functions

Function	Description
preg_match(pattern, string [, captured]) Used for input validation	Perform a regular expression match and will return 0 or 1, indicating whether or not a match is found. If found, the last argument will contain the first match.
preg_match_all(pattern, string, matches [, order])	Perform a global regular expression match and will return the no. of matches made or FALSE if none found. The function takes an optional ordering constant
preg_replace(pattern, replacement, subject [, limit])	Perform a regular expression search and replace. Limit is used for limiting the no. of replacements
preg_split(pattern, string [, limit [,flags]])	Split string by a regular expression

Examples:

Regex	Matches
`/^0\d{2}-\d{7,8}$/`	Malaysian mobile phone numbers: • **012-1234567** • **012-12345678**
`/^\d{2}[A-Z]{3}\d{5}$/`	TARUMT student id numbers: • **12XXX12345**
`/^image\/(jpeg|png)$/i`	File type: • **image/jpeg** • **image/png** **i** = case-insensitive matching
`/^.+\.(jpg|png)$/i`	File name: • **x.jpg** • **x.png** **i** = case-insensitive matching